

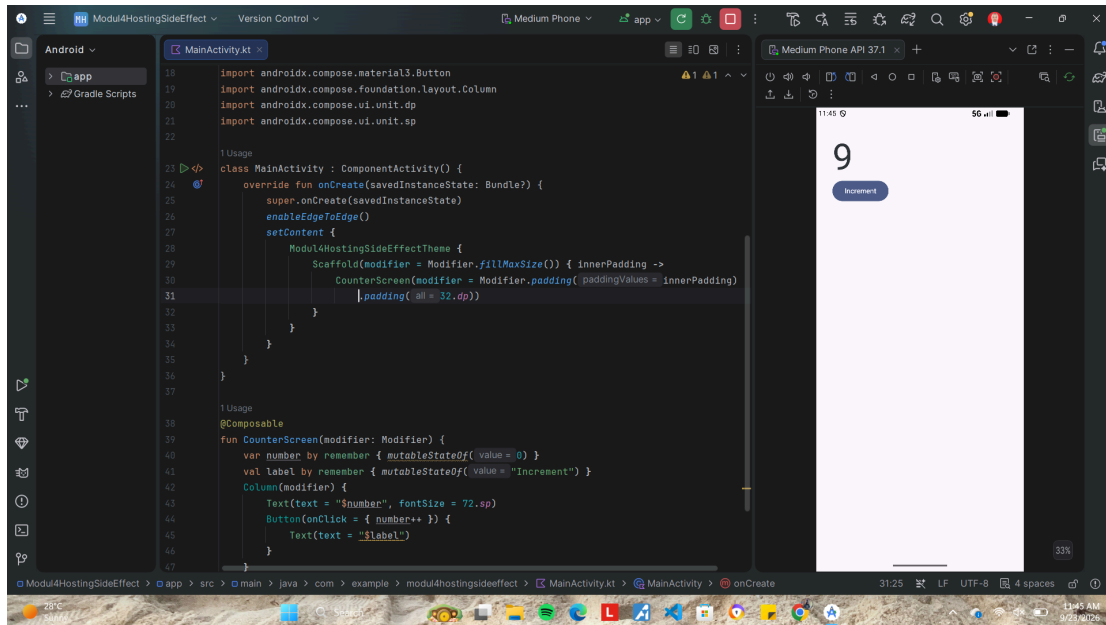
MODUL 4: STATE HOISTING, PERUBAHAN KONFIGURASI, DAN SIDE-EFFECT

4.5 Percobaan

4.5.1 State Hoisting

Aktivitas Percobaan

1. Tampilan aplikasi setelah tombol “Increment” di klik beberapa kali:



Gambar 1.

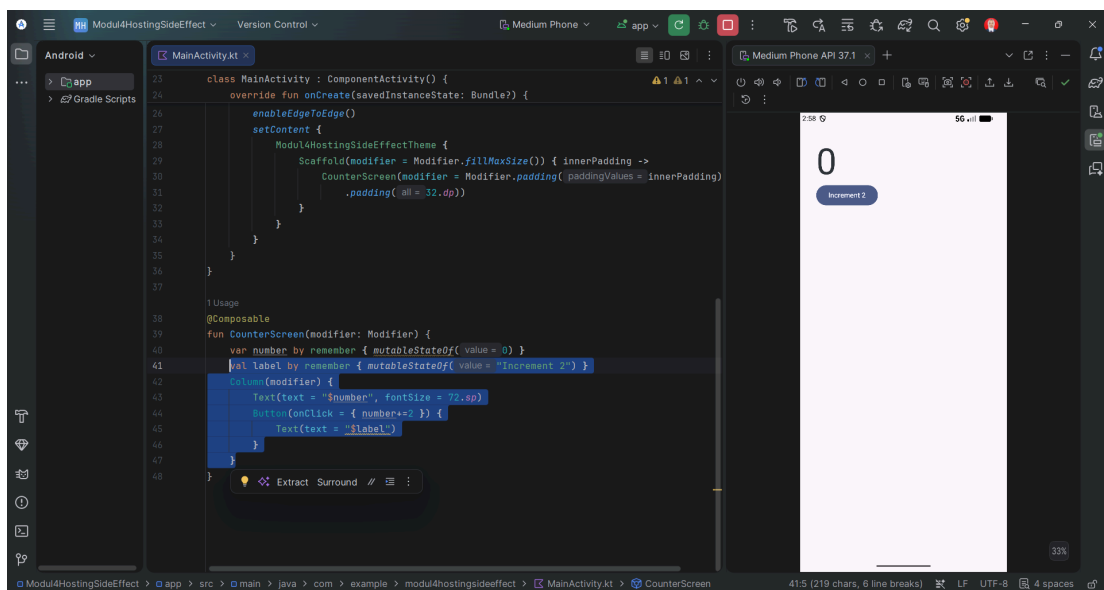
- a. Setiap kali tombol “Increment” di klik, angka yang ditampilkan di layar bertambah 1. Misalnya angka awalnya adalah 0, setelah di klik sekali maka angka yang ditampilkan menjadi 1, ketika diklik lagi menjadi 2, 3, dan seterusnya.
- b. Hal itu terjadi karena setiap kali tombol “Increment” ditekan, kode `number++` akan dijalankan sehingga nilai `number` bertambah 1. Lalu karena `number` disimpan sebagai state, perubahan nilainya membuat tampilan ikut diperbarui. Jadi, angka pada `Text` akan berubah mengikuti nilai `number` setiap kali tombol ditekan.
- c. Variabel state yang berperan pada Composable `Text` yang menampilkan perubahan angka pada `Text` tersebut adalah `number`, yang ada pada baris kode `var number by remember { mutableStateOf(0) }`. Variabel `number` digunakan pada `Text` melalui baris kode `Text(text = "$number", fontSize = 72.sp)`. Sehingga ketika nilai `number` berubah, angka yang ditampilkan pada `Text` juga ikut berubah.
- d. Variabel state yang berperan untuk menampilkan label teks pada Composable `Button` “Increment” adalah `label`, yang ada pada baris kode `val label by remember { mutableStateOf("Increment") }`. Kemudian digunakan pada `Text` di dalam `Button`, pada baris kode `Text(text = "$label")`. Nilai awal yang diberikan pada `label` adalah tulisan “Increment”, sehingga tulisan yang muncul pada tombol adalah Increment.

2. a. Perubahan yang perlu dilakukan untuk mengubah agar nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap diklik) adalah dengan mengubah fungsi *CounterScreen* khususnya bagian *onClick* pada *Button number++* menjadi *number += 2*. tepatnya pada kode berikut:

```
Button(onClick = { number+=2 }) {
    Text(text = "$label")
}
```

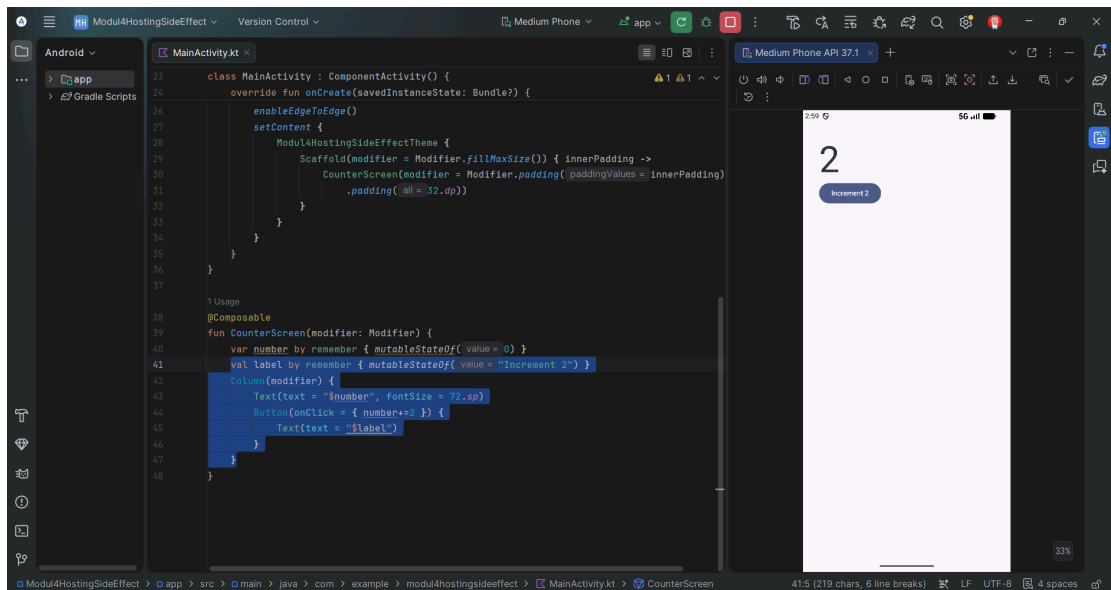
Kemudian, agar label pada button bertuliskan "Increment 2" dilakukan perubahan nilai pada variabel label di kode *val label by remember { mutableStateOf("Increment 2") }* dari yang awalnya "Increment" menjadi "Increment 2".

- b. Tampilan aplikasi sebelum tombol "Increment 2" di klik:



Gambar 2.

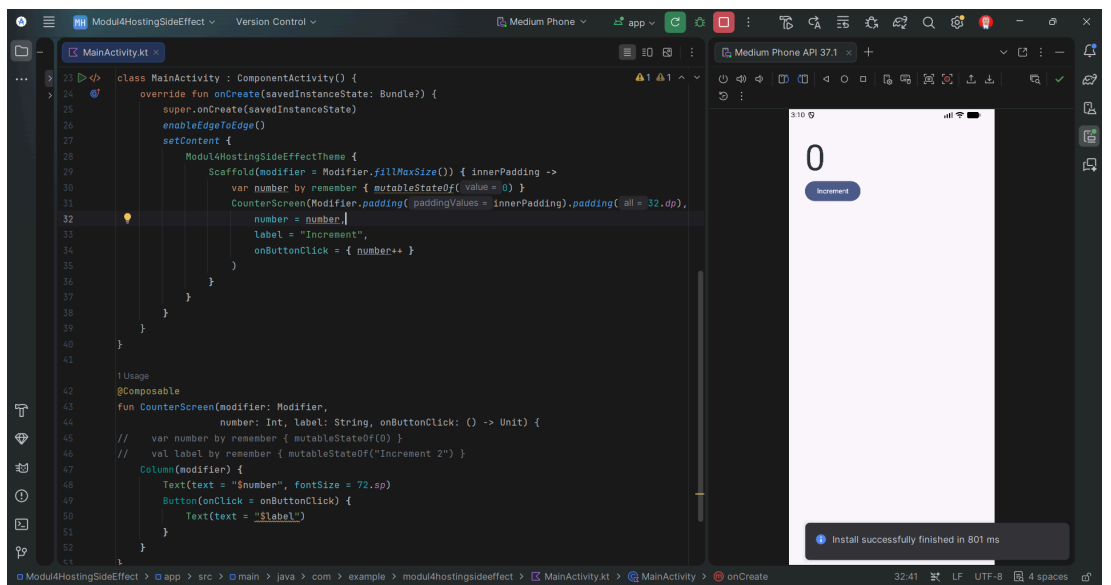
Tampilan aplikasi setelah tombol “Increment 2” di klik sekali:



Gambar 3.

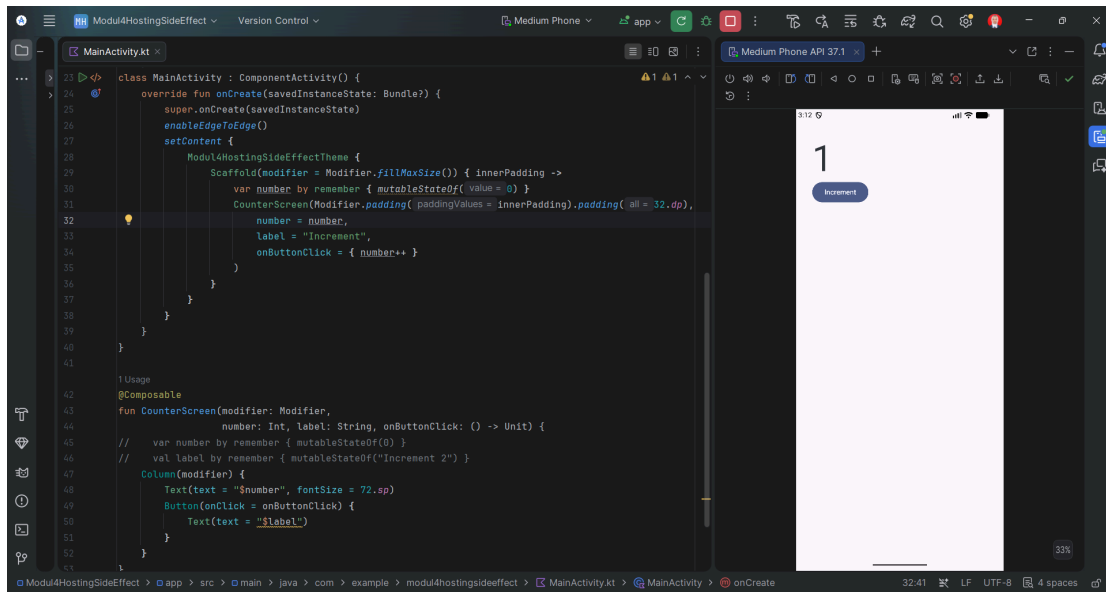
3. Melakukan *state* dan *event hoisting* pada state dan event yang sebelumnya terdapat di dalam Composable *CounterScreen*.

4. Tampilan aplikasi sebelum tombol “Increment” di klik:



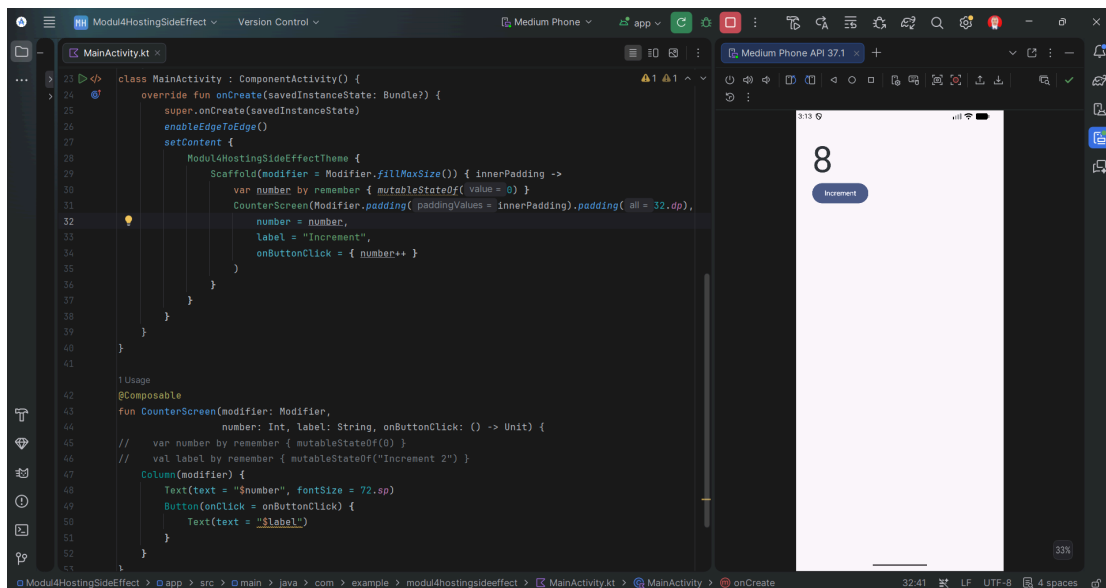
Gambar 4.

Tampilan aplikasi setelah tombol “Increment” di klik sekali:



Gambar 5.

Tampilan aplikasi setelah tombol “Increment” di klik beberapa kali:



Gambar 6.

- Program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap sebelumnya. Angka pada tampilan layar tetap bertambah 1 setiap kali tombol increment di klik.
- Proses increment saat ini dilakukan pada lambda `onButtonClick` saat `CounterScreen` dipanggil di dalam `onCreate()`, yaitu pada kode `onButtonClick = { number++ }`. Sehingga `CounterScreen` hanya menerima fungsi tersebut sebagai parameter dan menjalankannya ketika tombol di klik.
- State `number` saat ini di deklarasikan di dalam `onCreate()`, yaitu pada kode `var number by remember { mutableStateOf(0) }`. Sedangkan `Text` dan `Button` tetap di deklarasikan di dalam Compose `CounterScreen`:

`Column(modifier) {`

```
Text(text = "$number", fontSize = 72.sp)
```

```
Button(onClick = onButtonClick) {  
    Text(text = "$label")  
}  
}
```

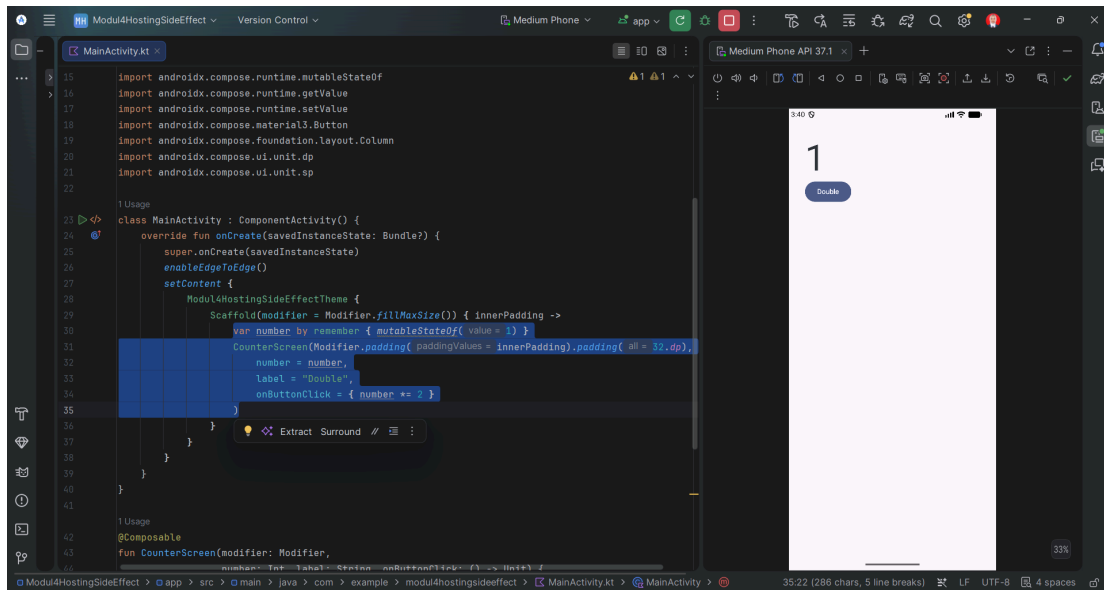
State berada di pemanggil, sedangkan tampilan UI tetap berada di *CounterScreen*.

5. a. Mengubah label tombol menjadi "Double", yaitu dengan mengubah pemanggilan *CounterScreen* bagian *label = "Increment"* menjadi *label = "Double"*.
b. Mengubah nilai awal *number* dari 0 menjadi 1 yaitu dengan mengubah baris kode *var number by remember { mutableStateOf(0) }* menjadi *var number by remember { mutableStateOf(1) }*.
c. Mengubah fungsi *onButtonClick* dari yang sebelumnya *onButtonClick = { number++ }* menjadi *onButtonClick = { number *= 2 }*. Sehingga setiap tombol ditekan, nilai *number* akan dikalikan 2.
d. Setelah perubahan yang dilakukan pada point a,b, dan c, kode state dan pemanggilan *CounterScreen* didalam *onCreate()* menjadi:
var number by remember { mutableStateOf(1) }

```
CounterScreen(  
    Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onButtonClick = { number *= 2 }  
)
```

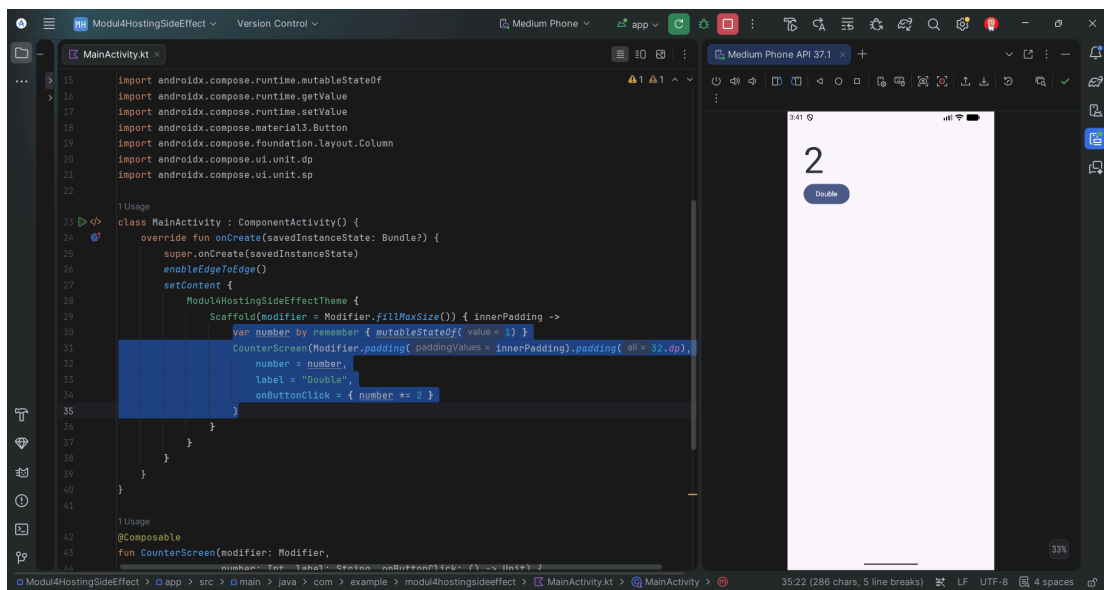
Dengan perubahan yang dilakukan pada nomor ini, nilai awal yang ditampilkan adalah 1, dan setelah tombol Double di klik, nilainya menjadi 2, kemudian 4, 8, dan seterusnya.

6. a. Tampilan aplikasi sebelum tombol "Double" di klik:



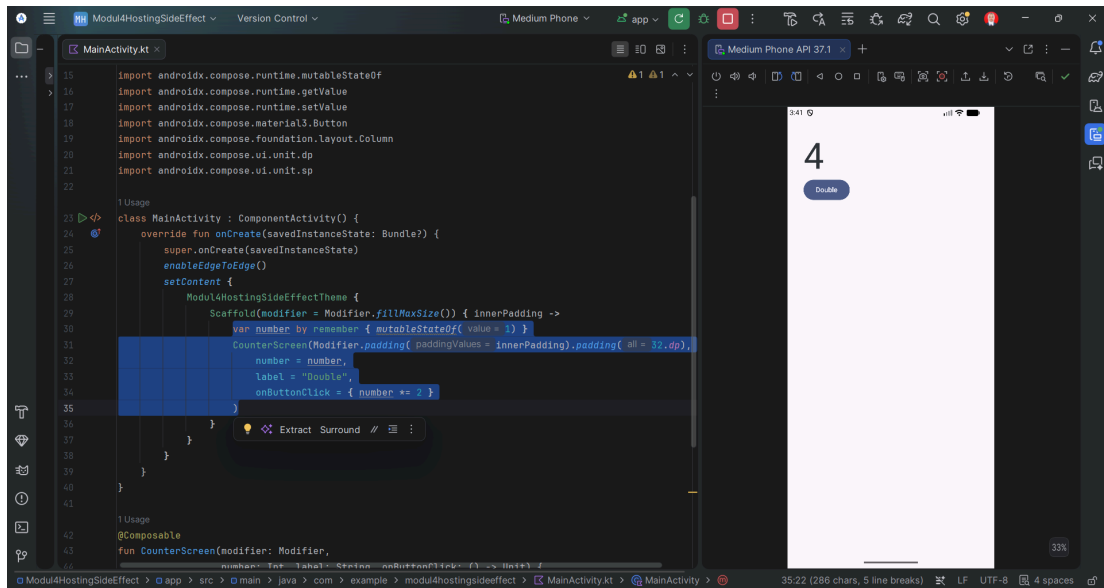
Gambar 7.

Tampilan aplikasi setelah tombol “Double” di klik sekali:



Gambar 8.

Tampilan aplikasi setelah tombol “Double” di klik 2 kali:



Gambar 9.

b. Perubahan dilakukan di dalam fungsi *onCreate()*, tepatnya pada saat state *number* dideklarasikan dan ketika *CounterScreen* dipanggil. Dimana pada tahap 7 kode programnya:

```
var number by remember { mutableStateOf(0) }
```

```
CounterScreen(
    Modifier.padding(32.dp),
    number = number,
    label = "Increment",
    onClick = { number++ }
)
```

Kemudian pada tahap 8 diubah menjadi:

```
var number by remember { mutableStateOf(1) }
```

```
CounterScreen(
    Modifier.padding(32.dp),
    number = number,
    label = "Double",
    onClick = { number *= 2 }
)
```

c. Tidak ada perubahan pada bagian utama *CounterScreen*. Struktur *CounterScreen* tetap menerima *number*, *label*, dan *onClick* sebagai parameter. Dengan kode program:

```
fun CounterScreen(
    modifier: Modifier,
    number: Int,
    label: String,
    onClick: () -> Unit
) {
    Column(modifier) {
        Text(text = "$number", fontSize = 72.sp)
    }
}
```

```

        Button(onClick = onButtonClick) {
            Text(text = "$label")
        }
    }
}

```

Perubahan nilai awal, tulisan tombol, dan proses perhitungan dilakukan pada bagian pemanggilan *CounterScreen* di *onCreate()*, bukan pada *composable* tersebut.

7. Dari percobaan ini, state hoisting membuat perubahan logika aplikasi bisa dilakukan tanpa harus mengubah isi dari *CounterScreen*. Setelah state dan event dipindahkan ke bagian pemanggil, *CounterScreen* hanya menerima data dan fungsi yang dibutuhkan untuk menampilkan UI.

Contohnya pada awalnya tombol digunakan untuk menambah angka:

```
onButtonClick = { number++ }
```

Kemudian ketika ingin diubah menjadi menggandakan angka, cukup diubah menjadi:

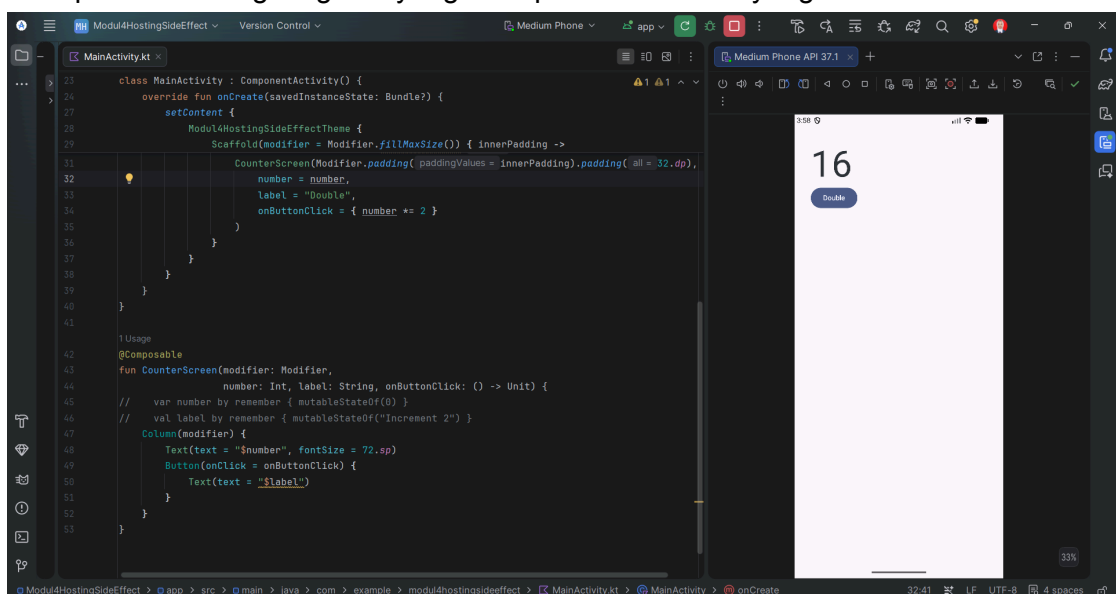
```
onButtonClick = { number *= 2 }
```

Begitu juga dengan label dan nilai awalnya. Bagian *CounterScreen* tidak perlu diubah lagi. Menurut saya, dari percobaan ini terlihat bahwa state hoisting membuat pengelolaan state dan tampilan UI menjadi terpisah. Jadi ketika logika program berubah, bagian yang perlu diubah bisa lebih terfokus tanpa mengubah keseluruhan *composable*.

4.5.2 Perubahan Konfigurasi

Aktivitas Percobaan

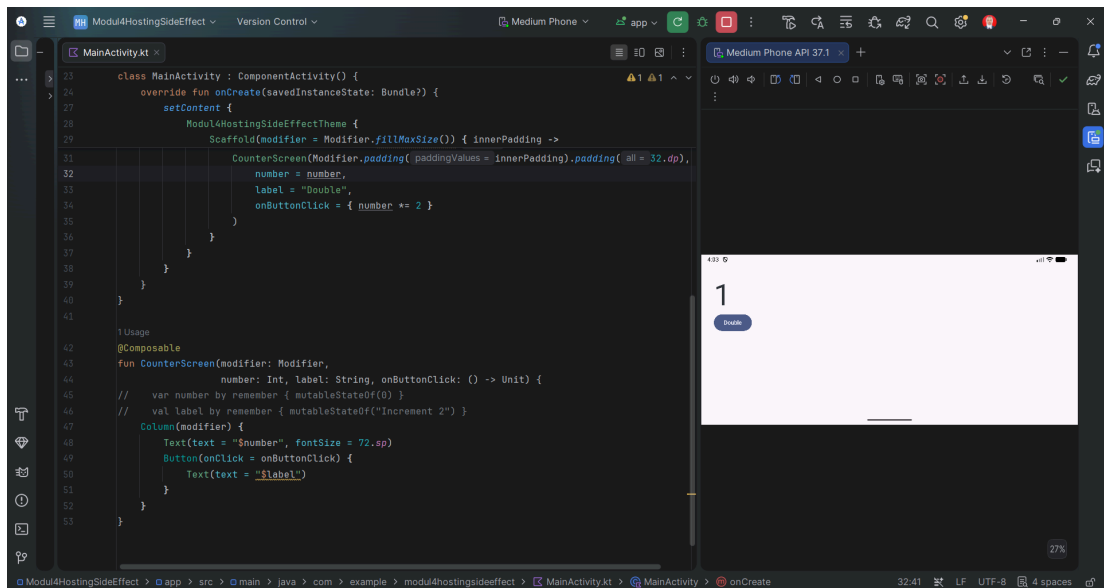
1. Tampilan aplikasi setelah tombol "Double" di klik beberapa kali sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset:



Gambar 10.

Program menampilkan angka 16 setelah saya menekan tombol sebanyak 4 kali, yaitu dari 1 -> 2 -> 4 -> 8 -> 16.

2. a. Setelah perangkat diputar dari posisi potrait ke landscape, nilai *number* berubah kembali ke nilai awal yaitu 1.



Gambar 11.

- b. Angka yang ditampilkan adalah 1. Hal ini terjadi karena pada kode masih menggunakan:

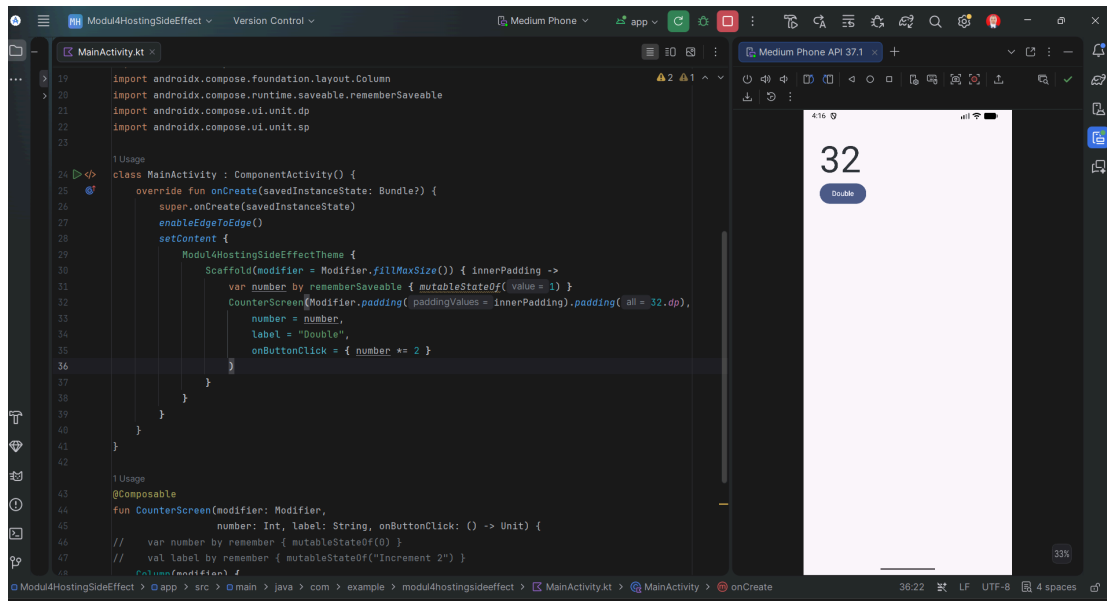
var number by remember { mutableStateOf(1) }

Saat layar diputar, terjadi perubahan konfigurasi sehingga Activity dibuat ulang lagi. Nilai yang sebelumnya disimpan dengan *remember* tidak ikut dipertahankan, sehingga *number* kembali menggunakan nilai awal, yaitu 1. *remember* juga hanya mempertahankan state selama recomposition dan tidak mempertahankannya ketika terjadi perubahan konfigurasi seperti melakukan rotasi layar.

Setelah itu kode diubah menjadi:

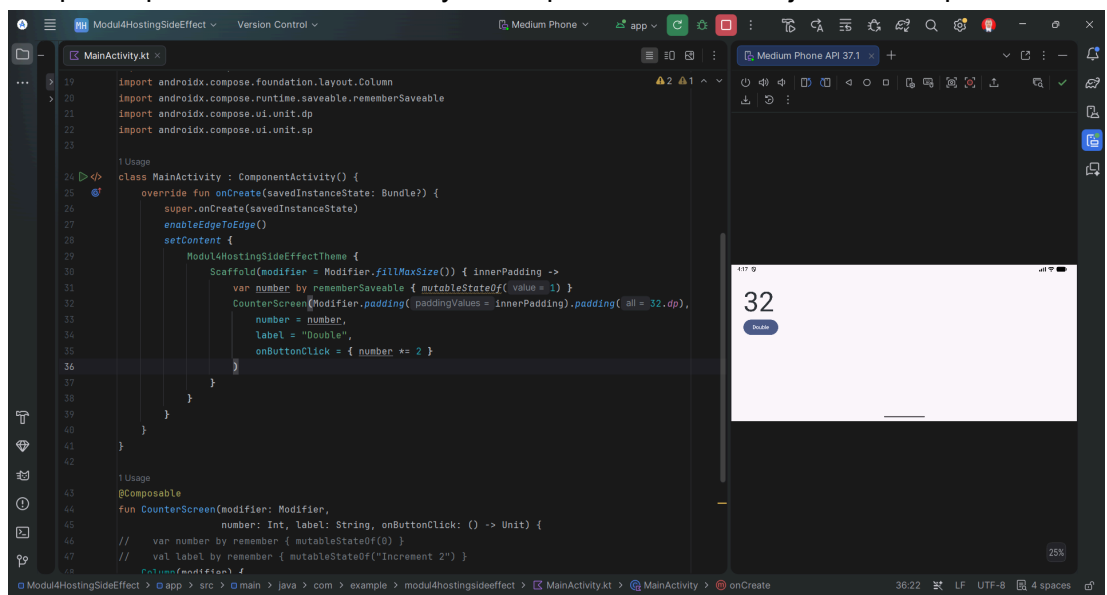
var number by rememberSaveable { mutableStateOf(1) }

Kemudian program dijalankan kembali pada posisi potrait dan tombol Double ditekan beberapa kali:



Gambar 12.

Tampilan aplikasi setelah rotasi layar dari potrait diubah menjadi landscape:



Gambar 13.

3. Menggunakan *rememberSaveable*

- Setelah tombol Double di klik beberapa kali, kemudian perangkat diputar dari potrait ke landscape, angka yang muncul tetap sama seperti angka sebelum layar diputar.
- Angka yang ditampilkan tetap pada nilai terakhir sebelum rotasi yaitu 32.
- Perbedaan *remember* dan *rememberSaveable* terlihat ketika layar diputar, dimana *remember* hanya menjaga nilai selama recomposition, sedangkan *rememberSaveable* juga menjaga nilai ketika terjadi perubahan konfigurasi.

4. Contoh lainnya yaitu pada aplikasi catatan. Misalnya pengguna sedang menulis catatan, lalu layar HP tidak sengaja diputar ke posisi landscape. Isi catatan yang sudah ditulis sebaiknya tetap ada dan tidak kembali kosong. Misalnya state teksnya dibuat menjadi seperti kode berikut:

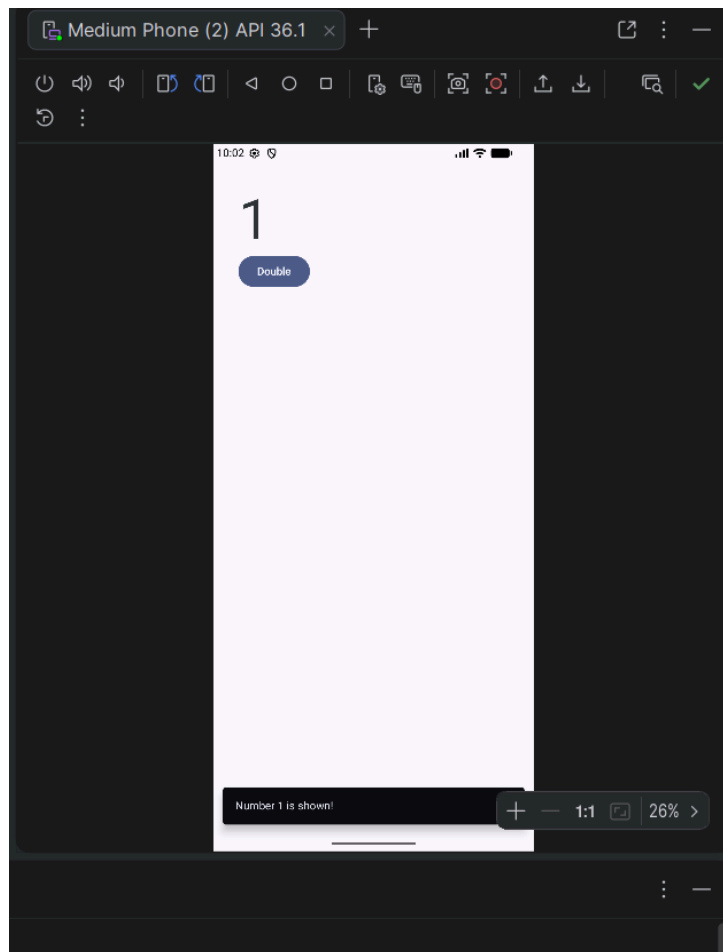
```
var note by rememberSaveable { mutableStateOf("") }
```

Jadi apabila pengguna sudah menulis beberapa kalimat kemudian layar diputar, isi catatannya tetap tersimpan. Tentunya ini cukup berguna karena pengguna tidak perlu mengetik ulang catatan yang sudah dibuat hanya karena posisi layar berubah.

4.5.3 LaunchedEffect

Aktivitas Percobaan 1

1. a. Saat pertama kali program dijalankan, angka awal yang tampil adalah 1 dan muncul Snackbar dengan pesan "Number 1 is shown!".



Gambar 14.

- b. Pesan tersebut dapat muncul padahal tidak ada input yang diberikan pada tampilan UI karena *LaunchedEffect(number)* langsung dijalankan saat program pertama kali masuk ke tampilan. Saat itu nilai *number* masih 1, jadi pesan yang ditampilkan juga mengikuti nilai tersebut. Jadi, Snackbar tidak harus menunggu ada

klik dari pengguna. Begitu *LaunchedEffect* dijalankan, kode di dalamnya juga langsung dijalankan.

c. Yang memicu *SnackBar* adalah *LunchedEffect(number)*, tepatnya pada bagian:

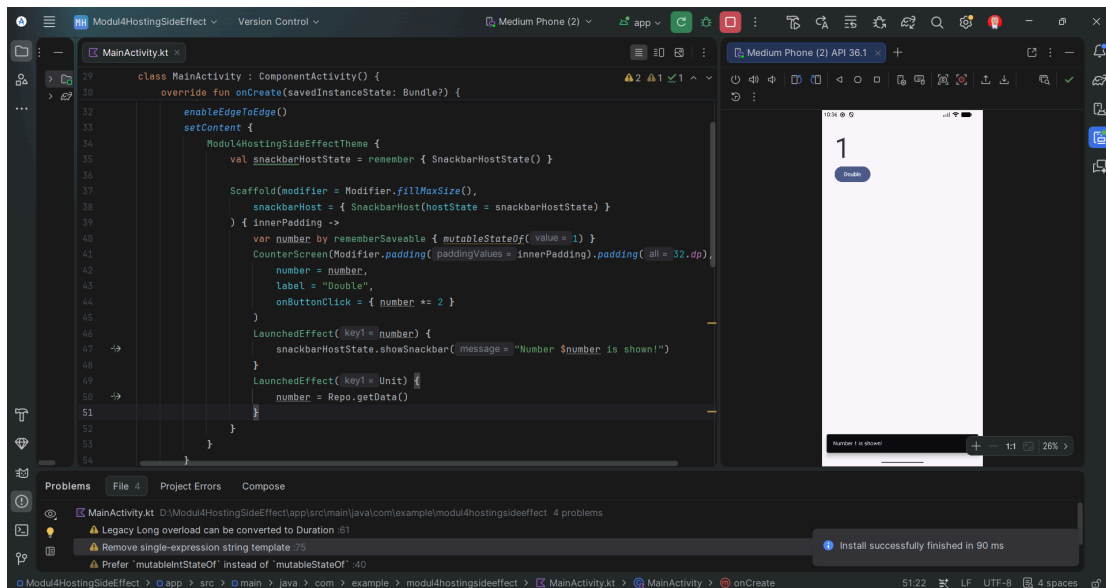
```
LaunchedEffect(number) {
    snackbarHostState.showSnackBar("Number $number is shown!")
}
```

Karena *number* dijadikan sebagai key, ketika nilai *number* berubah, *LunchedEffect* akan dijalankan lagi.

2. a. Setelah tombol Double di klik satu kali, angka yang awalnya 1 berubah menjadi 2, lalu muncul juga *SnackBar* dengan tulisan "Number 2 is shown!".
- b. Hal ini dapat terjadi karena saat tombol Double ditekan, kode *onButtonClick = { number *= 2 }* membuat nilai *number* yang awalnya 1 menjadi 2. Setelah nilainya berubah, *LaunchedEffect(number)* ikut berjalan lagi karena key yang digunakan adalah *number*. Kemudian kode didalamnya menjalankan: *snackbarHostState.showSnackBar("Number \$number is shown!")*. Karena nilai *number* sekarang sudah 2, *SnackBar* menampilkan "Number 2 is shown!".

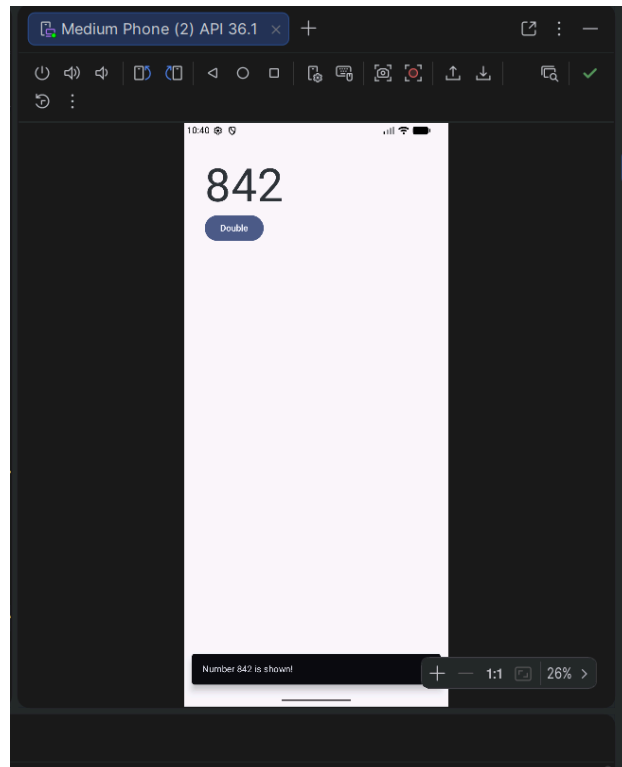
Aktivitas Percobaan 2

1. a. Saat pertama kali program dijalankan, angka awal yang tampil adalah 1, karena nilai awal *number* masih diatur menjadi 1. *SnackBar* juga akan menampilkan pesan "Number 1 is shown!" dari *LaunchedEffect(number)*.



Gambar 15.

- b. Setelah menunggu sekitar 2 detik, nilai angka pada layar berubah menjadi angka acak antara 100 sampai 999. Angka tersebut berasal dari kode *return Random.nextInt(100, 1000)*. Sehingga hasilnya berbeda setiap kali program dijalankan. Lalu waktu tunggu selama 2 detik tersebut berasal dari *delay(2000)* yang digunakan untuk mensimulasikan proses mengambil data dari internet.



Gambar 16.

Setelah *number* berubah, Snackbar juga akan menampilkan pesan sesuai angka baru tersebut. Misalnya seperti pada gambar 16, angka yang saya dapat adalah 842, maka Snackbar juga akan menampilkan pesan "Number 842 is shown!".

c. Perubahan angka dipicu oleh *LaunchedEffect(Unit)* yang menjalankan:

```
LaunchedEffect(Unit) {
    number = Repo.getData()
}
```

Dimana *Repo.getData()* membutuhkan waktu 2 detik karena ada *delay(2000)*. Setelah selesai, hasil angka acak dikembalikan dan dimasukkan ke variabel *number*. Ketika *number* berubah, *LunchedEffect(number)* dari percobaan sebelumnya ikut berjalan kembali. Berikut adalah potongan kode program yang berperan dalam menampilkan pesan baru pada Snackbar:

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}
```

2. a. Perbedaan antara *LunchedEffect(state)* dengan *LunchedEffect(Unit)* ada pada key yang digunakan. Pada *LaunchedEffect(number)* yang menjadi key adalah *number*. Jadi ketika nilai *number* berubah, effect akan dijalankan kembali. Sedangkan *LaunchedEffect(Unit)* menggunakan *Unit* sebagai key, Pada percobaan ini, bagian tersebut digunakan untuk menjalankan proses *Repo.getData()* ketika program mulai dijalankan.
- b. *LaunchedEffect(number)* dijalankan saat pertama kali tampil dan akan berjalan lagi setiap kali nilai *number* berubah. Jadi ketika angka dari 1 berubah menjadi angka hasil *Repo.getData()*, effect ini dijalankan lagi untuk menampilkan Snackbar.

Sedangkan *LaunchedEffect(Unit)* dijalankan ketika pertama kali masuk ke komposisi. Karena key nya tetap Unit, perubahan nilai *number* tidak membuat bagian ini dijalankan lagi. Jadi *Repo.getData()* pada percobaan ini cukup dijalankan saat awal program.